

Arquitectura e integración del editor de texto CLI

Hecho por / Grupo:

Thomas Serna Saldarriaga - 4328

Mateo Montoya Ospina - 4627

Juan Esteban Palacio Betancur - 4328

Proyecto:

Editor de texto con implementación de SysCalls en Linux

Asignatura:

Sistemas operativos

Universidad:

EAFIT

Profesor:

Edison Valencia Díaz

Resumen del proyecto

Este documento detalla la arquitectura, decisiones de diseño y todas las estrategias implementadas a lo largo de la creación del proyecto para permitir un editor de texto interactivo operado por interfaz de línea de comandos (CLI), enfocado en sistemas Unix/Linux.

El proyecto, debido a los requerimientos indicados en el proyecto, se trabajó con el lenguaje de programación C. Ya que este lenguaje utiliza indicaciones de bajo nivel, las cuales son requeridas para recrear una experiencia lo más cercana posible a la creación de un editor de texto, la cual funcione por medio de una interacción directa con el kernel de Linux.

A lo largo del proyecto se implementan diversas Syscalls, tales como: open, read, write, lseek, ftruncate, close. Todas pertenecientes al estándar POSIX; de este modo nos desligamos del uso de herramientas previamente implementadas por el lenguaje C, como lo sería <stdio.h>, y simulamos esas funciones buscando una gestión eficiente tanto de memoria como de operaciones para hacer operaciones I/O en las operaciones tipo CRUD y demás de archivos de texto.

Integración estratégica con el shell educativo y la creación del editor del texto

Tras haber revisado lo pedido en el proyecto, se decidió la implementación de la Shell educativa provista por el profesor y, por medio de la estructura en el repositorio ya implementada, se procedió a simular la arquitectura tanto del código como de carpetas para, de este modo, mantener las buenas prácticas de código, facilitar la creación del código al mantener un mismo estándar de lo que ya se tenía con lo que se quería desarrollar y la interconexión de los distintos archivos que se van desarrollando.

Respecto a la interconexión en la parte de ejecución del editor de texto, se decidió que, para mantener la congruencia entre el acceso al editor de texto y la función de la shell en este proceso, se decidió que el editor de texto será una función accesible por medio de la shell educativa, tras haberla ejecutado y haber ejecutado el comando "editar". También, para que la shell no sea únicamente un paso extra para acceder al editor, se decidió dejar las funciones básicas que ya venían en ella previamente.

Justificación de las decisiones tomadas a partir del pensamiento crítico

1. Aislamiento de procesos: Debido a la naturaleza propia del editor de texto, que consiste en un manejo de memoria dinámica, punteros y descriptores de archivos que son alterados constantemente, se decidió aislar el proceso para que, en caso de un memory leak o un segmentation fault, no se caiga toda la shell.

2. Separación de responsabilidades: Para conservar la escalabilidad y mantenibilidad del editor de texto, todos los comandos están separados en distintos archivos según su funcionalidad y relación.

3. Manera de invocación: Se utiliza un ciclo de vida propio, estilo REPL secundario, para que, de este modo, sea bloqueada la interacción con la shell por parte del usuario hasta que se ingrese el comando de salida (q). De este modo, se aísla el proceso, asegurando la estabilidad de la shell, facilitando la inyección de descriptores de archivos, y se respeta el principio fundamental de las buenas prácticas de código de única responsabilidad.

4. Uso de GitHub y flujo de commits: Se decidió implementar el gitignore para evitar la carga de archivos innecesarios o que se generan al momento de la compilación, como el tipo .o, y mantener las buenas prácticas al usar GitHub. Además, se utiliza un archivo tipo auto approve para que, de este modo, se mejore el flujo al momento en el cual todos los miembros fuéramos subiendo archivos y actualizando el documento.

5. Conservar el uso makefile: Se decidió conservar el uso de makefile como medio para facilitar la compilación del código y permitiendo un óptimo testeo del código a medida que se agregaban funcionalidades.

6. Estructura en la cual funciona el editor de texto: Debido a que el proyecto mencionaba en sus primeras instancias el enfoque en uso de call systems, CLI y no usar concurrencia, se decidió usar un editor de texto el cual no contiene estructuras intermedias al momento de cargar el archivo para sus funciones CRUD, tales como lo serían listas enlazadas, árboles, etc. Sino que por medio de puros syscalls se envían todos los comandos e interacciones con el archivo deseado, en vez de hacerlo por pedazo, lo cual reduce sustancialmente la velocidad de ejecución y la seguridad de las acciones que se ejecutan al archivo, pero que, de este modo, se apega con las condiciones que menciona el archivo y ofrece un desafío al hacernos buscar las mejores maneras de optimizar los procesos para reducir la ya gran latencia de los procesos y el manejo de errores durante la ejecución del proceso para, de este modo, ofrecer una alternativa fluida y disfrutable para el usuario.

7. Comentarios: Se decidió que, para los comentarios dentro del repositorio, siguieran todos la misma estructura para las partes importantes, teniendo el nombre y la descripción de cómo funcionan y qué implementan para así mantener la legibilidad de lo que hace el proyecto. Además del uso de comentarios simples de una sola línea para uso de syscalls específicos, para así saber qué es lo que hacen o indicar el objetivo de esos elementos en concreto.

8. README: Se enfoca en la parte de ejecución y los elementos más básicos de cómo utilizar el editor de texto y la shell, permitiendo que los usuarios puedan usarla rápidamente. Mientras que este archivo ofrece una perspectiva introspectiva de las decisiones tomadas y las decisiones técnicas tomadas se ubican en el código mismo.

Arquitectura interna del editor y modularidad

Este tipo de proyectos tienden a convertirse en código spaghetti y un tumulto de funciones en un solo archivo, por lo que se enfocó en implementar un diseño modular dividido en subdirectorios en src/editor/, conteniendo cada uno los siguientes elementos y objetivos:

- editor_basic.c: Contiene el ciclo de vida del archivo y las funciones de apertura (o) y salida (q).

- editor_inspection.c: Aísla los comandos de lectura estricta y metadatos (p, m, s), garantizando que no modifiquen accidentalmente el archivo.
- editor_modify.c: Agrupa los comandos de escritura y eliminación (a, i, d), donde recaen los retos algorítmicos más fuertes sobre el desplazamiento de bytes.
- editor_clipboard.c: Encapsula la lógica estricta de copiado y pegado local (y, x).

Retos técnicos y uso de Sys Calls dentro del proyecto

A lo largo del proyecto, debido a la multitud de funciones y la necesidad de implementar syscalls a lo largo de esta, se fueron usando distintos elementos con distintos objetivos, los cuales se mencionan en este apartado.

Comandos Base y Restricción de I/O

- Apertura (comando o): Se utiliza `open(filepath, O_RDWR | O_CREAT, 0644)`, obteniendo un File Descriptor (FD). Se asignaron permisos base seguros a los archivos generados.
- Inserción (comando a): Se hace uso de la llamada `lseek(fd, 0, SEEK_END)` para reposicionar el puntero del Kernel al final del archivo antes de inyectar bytes, que simbolizan los nuevos textos, con `write()`, previniendo la corrupción o alteración no deseada de los datos existentes.
- Borrado de línea (comando d): Este fue un desafío bastante grande en sí debido a que C no posee Syscalls enfocadas directamente a esto, por lo que se tomó otro enfoque. El reto se abordó ubicando los offsets de la línea a borrar, subiendo a memoria todo el contenido inferior mediante un búfer dinámico, reposicionando el cursor con `lseek()` en el inicio de la línea eliminada y rescribiendo el búfer. Finalmente, para eliminar los bytes sobrantes en la cola del archivo, fue muy importante el uso de `ftruncate()`, modificando la longitud del inodo.

Retos de Escalamiento Nivel 2: Inserción y Búsqueda

- Inserción arbitraria (comando i): Requiere un algoritmo de funcionamiento inverso al del borrado. Se debió usar búferes dinámicos de manera precavida (teniendo como objetivo prevenir fugas de memoria) para desplazar de forma segura todos los bytes posteriores hacia abajo, haciendo el espacio necesario para la nueva línea antes de que termine el archivo.
- Búsqueda (comando s): Se implementó cargando líneas a memoria, buscando coincidencias de la cadena ingresada e imprimiendo solo los offsets coincidentes, retornando siempre el cursor del archivo a su lugar de origen.

Retos de Escalamiento Nivel 3: Metadatos y Portapapeles

- Metadatos del Archivo (comando m): Se utilizó la llamada al sistema `fstat()` para recuperar una estructura `stat` directamente desde el Kernel de Linux. Adicionalmente, se utilizó el syscall `getuid()` (requerido en la tabla de syscalls del proyecto). Esto facilitó consultar a bajo nivel el tamaño en disco, los permisos POSIX y la información vinculada al propietario del inodo.

- Copiar y Pegar (comandos y / x): Para implementar el portapapeles se extrajo el bloque de memoria seleccionado en y a un almacenamiento temporal persistente, el cual es almacenado por el editor de texto, y luego reutilizado con la misma lógica de inserción de búfer para el comando x. Se ha de destacar que en la arquitectura del sistema se referenció la system call link(), abriendo la puerta a manejos eficientes de referencias a nivel de sistema de archivos sin necesidad de duplicar cargas pesadas de I/O.

5. Manejo de Errores

Se implementó un robusto manejo de excepciones mediante un enfoque de programación defensivo, el cual venía propuesto de manera previa por el programa del shell educativo:

- Ninguna llamada a disco es automáticamente asumida exitosa; todas las ejecuciones de open(), read() o write() validan si el retorno es -1.
- Cuando el Kernel detecta excepciones de I/O (por ejemplo, intentar abrir un archivo bloqueado o ruta inválida), el editor maneja el escenario limpiamente, evitando los conocidos Segmentation Faults y emitiendo advertencias por la terminal, sin cerrar el Shell principal.

Testeos

Más allá de que este fuese un requerimiento dentro del archivo original, los testeos implementados desde un inicio en el programa fueron de una gran ayuda, ya que facilitaban la verificación de funcionalidades y el flujo de múltiples funciones a la vez.

Conclusión

Este proyecto nos demostró cómo herramientas tan comunes como un editor de texto hecho con componentes básicos del propio sistema operativo del dispositivo depende de una compleja sincronización y entrelazamiento de operaciones de bajo nivel (punteros a desplazamientos, estructuras stat y llamadas lseek y ftruncate). La decisión de dividir el proyecto en pequeños pedazos, cada uno con un objetivo en concreto y una determinada función, fue fundamental para el entendimiento de lo que estuvimos haciendo a lo largo del proyecto. El buen uso de prácticas de codificación fue esencial para llevar a cabo este proyecto, ya que redujo enormemente el tiempo invertido al momento de mantener el código, solucionar errores e implementar nuevas funciones, demostrando cómo mantenerse a la par con los estándares de la industria reduce el peso a lo largo del proyecto. Y, por último, este proyecto nos ayudó a mejorar el trabajo colaborativo, uso de programas en ambientes Linux y manejo de consolas, y a entender la gran interrelación entre el kernel, las syscalls y los lenguajes de bajo nivel en herramientas tan básicas para un sistema operativo.

Link al repositorio (Github): [ThomasSerna/shell-text-editor](https://github.com/ThomasSerna/shell-text-editor)

Link al video: [video](#)